07. OPinit 분쟁 해결 프로세스 분석

개요

OPinit의 분쟁 해결 프로세스는 Optimistic Rollup의 핵심 보안 메커니즘으로, 잘못된 상태 전환이나 부정확한 output root에 대해 challenger가 이의를 제기하고 해결하는 시스템입니다. 이 문서는 challenger와 proposer 간의 상호작용, 거버넌스 기반 분쟁 해결 메커니즘, 그리고 실제 분쟁 시나리오를 상세히 분석합니다.

분쟁 해결 시스템 아키텍처

주요 참여자

1. **Proposer (제안자)**

- L2 output root를 L1에 제출하는 역할
- `BridgeConfig.Proposer` 필드에 정의된 주소만 제안 권한 보유
- 정기적으로 L2 상태를 요약한 output root 제출

2. **Challenger (도전자)**

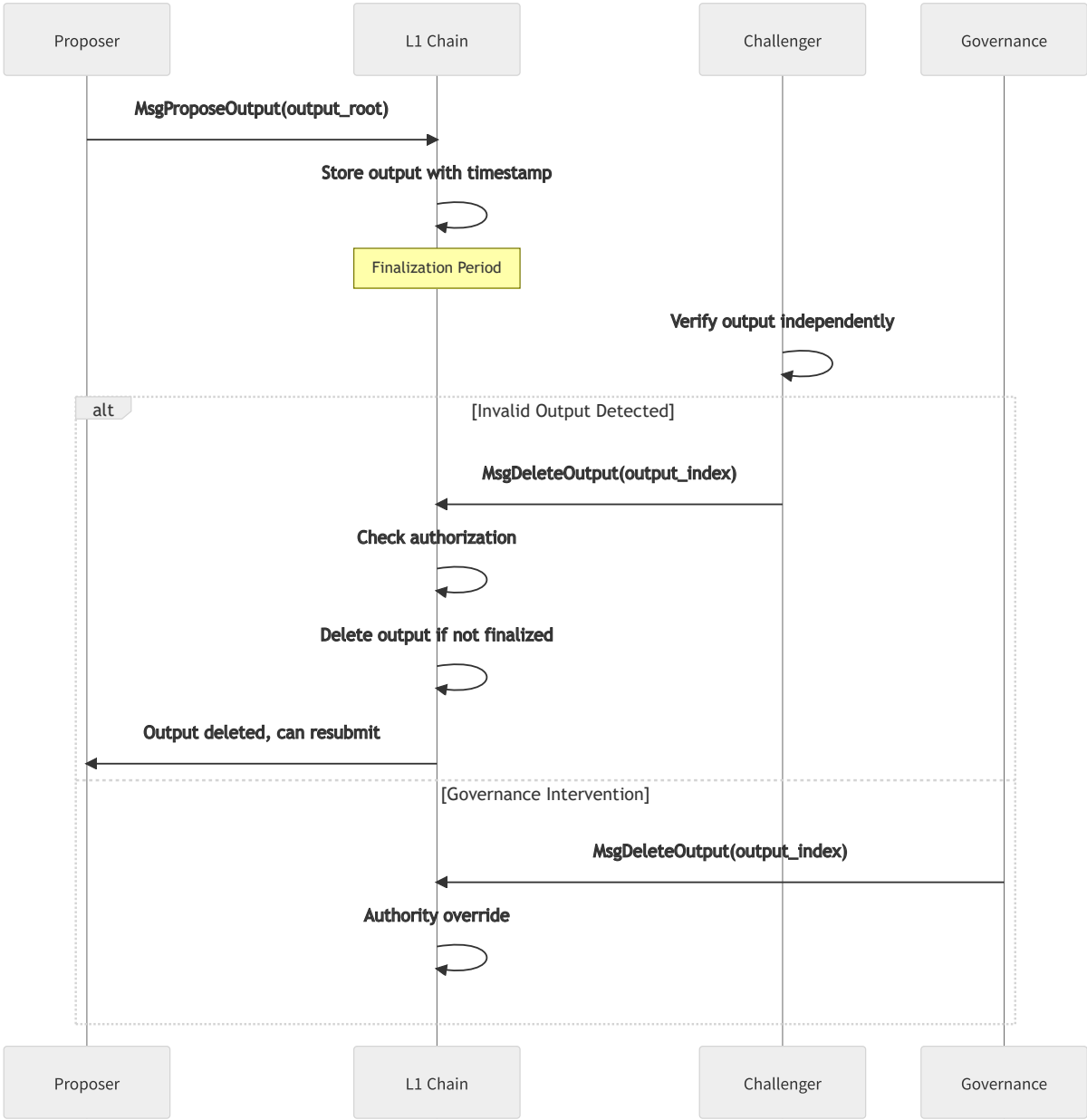
- 잘못된 output root를 감지하고 삭제하는 역할
- `BridgeConfig.Challenger` 필드에 정의된 주소가 도전 권한 보유
- L2 노드를 실행하여 독립적으로 상태 검증

3. **Authority (거버넌스)**

- 최종 중재자 역할 수행
- 일반적으로 x/gov 모듈 계정

- 복잡한 분쟁 상황에서 최종 결정권 보유

분쟁 해결 메커니즘


Proposer
L1 Chain
Challenger
Governance
MsgProposeOutput(output_root)
Store output with timestamp
Finalization Period
Verify output independently
alt
[Invalid Output Detected]
MsgDeleteOutput(output_index)
Check authorization
Delete output if not finalized
Output deleted, can resubmit
[Governance Intervention]
MsgDeleteOutput(output_index)
Authority override
Proposer
L1 Chain
Challenger
Governance

핵심 메시지 타입 분석

MsgProposeOutput

Proposer가 L2 output root를 L1에 제출하는 메시지입니다.

```
type MsgProposeOutput struct {
Proposer  string  // 제안자 주소
BridgeId  uint64  // 브리지 ID
OutputIndex  uint64  // Output 인덱스
L2BlockNumber  uint64  // L2 블록 번호
OutputRoot []byte  // Output root 해시 (32바이트)
}
```

권한 검증 로직:

```
// permission check
if  proposer != bridgeConfig.Proposer {
return  nil, sdkerrors.ErrUnauthorized.Wrap("invalid proposer")
}
```

순차성 검증:

```
// fetch next output index
outputIndex, err := ms.IncreaseNextOutputIndex(ctx, bridgeId)
if  outputIndex != req.OutputIndex {
```

```
return nil, types.ErrInvalidOutputIndex.Wrapf("expected %d, got %d", outputInde
}
```

MsgDeleteOutput

Challenger가 잘못된 output을 삭제하는 메시지입니다.

```
type MsgDeleteOutput struct {
Challenger string // 도전자 주소
BridgeId uint64 // 브리지 ID
OutputIndex uint64 // 삭제할 output 인덱스
}
```

권한 검증 로직:

```
// gov, current proposer or current challenger can delete output.
if ms.authority != challenger &&
bridgeConfig.Proposer != challenger &&
bridgeConfig.Challenger != challenger {
return nil, sdkerrors.ErrUnauthorized.Wrapf(
"invalid challenger; expected %s, %s or %s, got %s",
ms.authority, bridgeConfig.Proposer, bridgeConfig.Challenger, challenger)
}
```

분쟁 해결 프로세스 상세 분석

1. Output Root 제안 단계

프로세스:

1. Proposer가 정기적으로 L2 상태를 집계하여 output root 생성
2. `MsgProposeOutput` 을 통해 L1에 제출
3. L1에서 권한 검증 및 순차성 확인
4. Output 저장 시 L1 블록 시간 기록

코드 구현:

```
// store output proposal
if  err := ms.SetOutputProposal(ctx, bridgeId, outputIndex, types.Output{
OutputRoot: outputRoot,
L1BlockNumber: uint64(sdkCtx.BlockHeight()),
L1BlockTime: sdkCtx.BlockTime(),
L2BlockNumber: l2BlockNumber,
});  err != nil {
return  nil, err
}
```

2. Challenger 모니터링 단계

모니터링 방식:

- Challenger는 독립적인 L2 노드를 실행

- 제출된 output root와 자신이 계산한 root 비교
- 불일치 발견 시 분쟁 개시

검증 로직:

```
func (k Keeper) isFinalized(ctx context.Context, bridgeId  uint64, output types.

bridgeConfig, err := k.GetBridgeConfig(ctx, bridgeId)

if  err != nil {

return false, err

}

return  k.isFinalizedWithConfig(ctx, bridgeConfig, output)

}


func (k Keeper) isFinalizedWithConfig(ctx context.Context, bridgeConfig types.Br

return  sdk.UnwrapSDKContext(ctx).BlockTime().Unix() >=

output.L1BlockTime.Add(bridgeConfig.FinalizationPeriod).Unix(), nil

}
```

3. 분쟁 개시 단계

분쟁 조건:

- Output이 아직 finalized되지 않은 상태
- Challenger가 유효한 권한을 보유
- 삭제하려는 output index가 유효한 범위 내

삭제 프로세스:

```go
// delete output proposals in [outputIndex, nextOutputIndex) range
for i := outputIndex; i < nextOutputIndex; i++ {
if err := ms.DeleteOutputProposal(ctx, bridgeId, i); err != nil {
return nil, err
}
}

// rollback next output index to the deleted output index
if err := ms.NextOutputIndexes.Set(ctx, bridgeId, outputIndex); err != nil {
return nil, err
}
```

4. Finalization Period 보안 메커니즘

시간 기반 보안:

- 각 output은 제출 후 `FinalizationPeriod` 동안 도전 가능
- Finalized된 output은 삭제 불가능
- 이는 충분한 검증 시간을 보장

Finalization 확인:

```go
func (k Keeper) DeleteOutputProposal(ctx context.Context, bridgeId, outputIndex
output, err := k.GetOutputProposal(ctx, bridgeId, outputIndex)
if err != nil {
return err
```

```
}

if isFinalized, err := k.isFinalized(ctx, bridgeId, output); err != nil {
return err
} else if isFinalized {
return types.ErrAlreadyFinalized
}

return k.OutputProposals.Remove(ctx, collections.Join(bridgeId, outputIndex))
}
```

거버넌스 기반 분쟁 해결

Authority 권한

거버넌스(Authority)는 다음과 같은 특별한 권한을 보유합니다:

1. **무조건적 삭제 권한**: Proposer나 Challenger와 관계없이 output 삭제 가능
2. **역할 변경 권한**: `MsgUpdateProposer`, `MsgUpdateChallenger` 를 통한 역할 재할당
3. **브리지 설정 변경**: Finalization period 등 핵심 매개변수 조정

거버넌스 개입 시나리오

1. 악의적 Challenger

```
// 거버넌스가 challenger 교체
```

```
func NewMsgUpdateChallenger(authority string, bridgeId uint64, challenger st
return &MsgUpdateChallenger{
Authority: authority,
BridgeId: bridgeId,
Challenger: challenger,
}
}
```

2. 악의적 Proposer

```
// 거버넌스가 proposer 교체
func NewMsgUpdateProposer(authority string, bridgeId uint64, newProposer str
return &MsgUpdateProposer{
Authority: authority,
BridgeId: bridgeId,
NewProposer: newProposer,
}
}
```

3. 긴급 상황 대응

```
// 거버넌스가 finalization period 조정
func NewMsgUpdateFinalizationPeriod(
```

```
authority  string,
bridgeId  uint64,
finalizationPeriod time.Duration,
) *MsgUpdateFinalizationPeriod
```

L1 검증자의 역할과 L2 노드 실행

L1 검증자의 책임

1. **거버넌스 참여**: 분쟁 해결 제안에 대한 투표
2. **L2 노드 실행**: 독립적인 L2 상태 검증
3. **합의 형성**: 올바른 상태에 대한 네트워크 합의

L2 노드 실행 과정

검증자의 L2 노드 실행:

```
// 검증자는 다음 정보를 바탕으로 L2 상태 재구성
type Output struct {
OutputRoot []byte  // 검증할 상태 루트
L1BlockNumber  uint64  // L1 블록 높이
L1BlockTime time.Time // L1 블록 시간
L2BlockNumber  uint64  // 해당 L2 블록 높이
}
```

검증 프로세스:

1. L1에서 배치 데이터 수집
2. L2 상태 전환 재실행
3. 계산된 상태 루트와 제출된 output root 비교
4. 불일치 시 거버넌스 제안 생성

실제 분쟁 시나리오와 해결 과정

시나리오 1: 정상적인 도전 과정

상황: Proposer가 잘못된 output root 제출

해결 과정:

1. **제출**: Proposer가 `MsgProposeOutput` 제출
2. **모니터링**: Challenger가 독립적으로 L2 상태 검증
3. **도전**: 불일치 발견 시 `MsgDeleteOutput` 제출
4. **삭제**: 권한 확인 후 output 삭제 및 index 롤백
5. **재제출**: Proposer가 올바른 output 재제출

코드 예시:

```
// Test case에서 확인할 수 있는 정상적인 도전 과정
func Test_DeleteOutput(t *testing.T) {
// Proposer가 output 제출
_, err = ms.ProposeOutput(ctx, types.NewMsgProposeOutput(proposerAddr, 1, 1, 100
require.NoError(t, err)
// Challenger가 output 삭제
_, err = ms.DeleteOutput(ctx, types.NewMsgDeleteOutput(challengerAddr, 1, 1))
```

```
require.NoError(t, err)

// Proposer가 올바른 output 재제출 가능

_, err = ms.ProposeOutput(ctx, types.NewMsgProposeOutput(proposerAddr, 1, 1, 100

require.NoError(t, err)

}
```

시나리오 2: Finalization 후 도전 시도

상황: Output이 이미 finalized된 후 도전 시도

해결 과정:

1. **제출**: Proposer가 output 제출
2. **시간 경과**: Finalization period 경과
3. **도전 시도**: Challenger가 삭제 시도
4. **실패**: `ErrAlreadyFinalized` 에러로 삭제 실패

코드 예시:

```
func  Test_DeleteOutputProposal(t *testing.T) {

// Finalization period 후 삭제 시도

err = input.OPHostKeeper.DeleteOutputProposal(

ctx.WithBlockTime(ctx.BlockTime().Add(time.Second*11)), 1, 1)

require.ErrorIs(t, err, types.ErrAlreadyFinalized)

// Finalization period 내 삭제는 성공

err = input.OPHostKeeper.DeleteOutputProposal(
```

```
ctx.WithBlockTime(ctx.BlockTime().Add(time.Second*9)), 1, 1)
require.NoError(t, err)
}
```

시나리오 3: 거버넌스 개입

상황: Challenger와 Proposer 간 지속적인 분쟁

해결 과정:

1. **분쟁 지속**: 반복적인 제출-삭제 사이클
2. **거버넌스 제안**: 커뮤니티가 해결 방안 제안
3. **투표**: L1 검증자들의 투표를 통한 결정
4. **실행**: 거버넌스 권한으로 최종 해결

권한 구조:

```
// 거버넌스는 proposer, challenger와 관계없이 삭제 가능
if ms.authority != challenger &&
bridgeConfig.Proposer != challenger &&
bridgeConfig.Challenger != challenger {
return nil, sdkerrors.ErrUnauthorized
}
```

시나리오 4: 악의적 행위자 대응

상황: Proposer 또는 Challenger의 악의적 행위

대응 방안:

1. **역할 교체**: 거버넌스를 통한 새로운 proposer/challenger 지정
2. **매개변수 조정**: Finalization period 연장으로 검증 시간 확보
3. **긴급 조치**: 브리지 일시 중단 및 복구

보안 고려사항

1. 시간 기반 보안

Finalization Period의 중요성:

- 충분한 도전 시간 제공
- 네트워크 지연 및 검증 시간 고려
- 너무 짧으면 보안 위험, 너무 길면 사용성 저하

2. 권한 관리

다층 권한 구조:

- Proposer: 제출 권한만 보유
- Challenger: 삭제 권한 보유
- Authority: 최종 중재 권한

3. 경제적 인센티브

스테이킹 메커니즘:

- Proposer와 Challenger의 경제적 이해관계
- 잘못된 행위에 대한 경제적 처벌
- 올바른 행위에 대한 보상

결론

OPinit의 분쟁 해결 프로세스는 다음과 같은 특징을 가집니다:

1. **다층 보안**: Challenger, Proposer, Governance의 3단계 보안 구조

2. **시간 기반 보안**: Finalization period를 통한 충분한 검증 시간 보장

3. **유연한 거버넌스**: 복잡한 상황에서의 커뮤니티 기반 해결

4. **투명한 프로세스**: 모든 분쟁 과정이 온체인에서 투명하게 진행

이러한 메커니즘을 통해 OPinit은 Optimistic Rollup의 보안성을 보장하면서도 효율적인 L2 운영을 가능하게 합니다. 분쟁 해결 프로세스는 악의적 행위자로부터 시스템을 보호하고, 정직한 참여자들이 안전하게 네트워크를 운영할 수 있는 환경을 제공합니다.